

# Multiplayer competitive type racing website

Project Proposal for UCS503P

Submitted to: Ms. Anushka

Thapar Institute of Engineering and Technology

Aargh Rai, CSED\*

Amitoj Singh, CSED<sup>†</sup>

Varun Bothra, CSED<sup>‡</sup>

Shivanjay, CSED<sup>§</sup>

August 21, 2026

## 1 Higher-order goal

... secondary framing

This project aims to make a claimed digital skill *verifiable*. Typing proficiency is routinely asserted on resumes and used as a screening filter for data-entry, transcription, support, yet the number itself is almost always self-reported or produced by a browser test that performs no server-side validation. While the broader topic is trustworthy skill assessment, the immediate engineering objective is to translate *verifiable, competitively-ranked skill measurement* into a measurable, deployable software solution.

## 2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by first shipping the core typing loop end-to-end (type → server-recorded result → visible score) and validating it with users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

## 3 Problem Statement

A user who wants to prove and improve their typing speed has no credible, competitive place to do so. Common pain points include:

- **Unverifiable results:** typical typing tests compute speed entirely in the browser, so a reported WPM can be produced by pasting, by a script, or by editing client state. The score carries no trust.

---

\*Roll No: 1024240004, Email: [arai.be24thapar.edu](mailto:arai.be24thapar.edu)

<sup>†</sup>Roll No: 1024240139, Email: [asingh56.be24thapar.edu](mailto:asingh56.be24thapar.edu)

<sup>‡</sup>Roll No: 1024240144, Email: [vbothra.be24thapar.edu](mailto:vbothra.be24thapar.edu)

<sup>§</sup>Roll No: 1024240074, Email: [sshivanjay.be24thapar.edu](mailto:sshivanjay.be24thapar.edu)

- **No meaningful competition:** practice tools are single-player. A user cannot race a skill-matched opponent, which removes the pressure conditions under which real performance is actually tested.
- **No persistent skill measure:** a single best-ever score is a noisy outlier, not a skill estimate. There is no rating that converges over many attempts the way chess ratings do.
- **Fragmented progress data:** attempts are not retained, so a user cannot see improvement over time or compare against peers at a similar level.

**Why this matters now (contextual relevance):** remote hiring and remote assessment have made screening filters more common while making them easier to fake. A low-cost, self-serve platform that produces a defensible speed figure is directly useful in exactly the hiring markets where such filters are applied.

## 4 Proposed Solution

... *Software Proposition*

### 4.1 Overview

Build a **web-based** competitive typing platform (“I See I Type”) with the following components:

- **Real-time race lobbies:** multiple players receive an identical prompt and see opponents’ live progress as they type.
- **Server-authoritative scoring:** speed and accuracy are computed from server-side timestamps, not from numbers the client reports.
- **Rating system:** an Elo-style rating updated after every rated race, giving each player a converging skill estimate rather than a best-score.
- **Fair-play validation:** bounded statistical checks on keystroke timing plus paste and impossible-rate detection; results are marked *verified* or *flagged*.
- **Contests and leaderboards:** scheduled events with standings and a post-contest rating update, plus global and mode-wise leaderboards.
- **Player profile:** race history, WPM/accuracy trend, and an activity heatmap.

### 4.2 Core workflow

1. Player signs in and either enters the quick-match queue or creates a lobby (rated or unrated).
2. The server groups players by rating proximity, distributes an identical prompt, and starts a synchronised countdown.
3. Clients stream typing progress; the server timestamps each update and broadcasts live positions to the lobby.
4. The race terminates on completion or timeout; the server computes WPM and accuracy from its own timestamps.
5. Fair-play checks run on the recorded keystroke trace, marking the result verified or flagged.
6. Ratings are updated, the result is written to history, and leaderboards and profiles reflect the change.

### 4.3 Operational constraints for an educational, resource-constrained setting

- Keep the solution usable on low-to-mid range devices via responsive web design and a keyboard-first interface.
- Tolerate unstable networks: progress is sent as periodic ticks with reconciliation, and a race degrades to unrated rather than failing outright if a connection is lost.
- Avoid dependence on advanced research algorithms; fair-play detection uses bounded statistical heuristics, not learned models.
- Design for deployability using common web hosting and managed databases.

## 5 Solution Approach

*... Engineering focus*

This is an engineering course project, so we focus on distributed state management, workflow and system correctness rather than novel algorithm research.

### 5.1 Web architecture

*... web-based solution*

A typical 3-tier web architecture, extended with a real-time channel:

- **Frontend:** an interactive typing surface with character-level feedback, live opponent progress, and lobby, contest and profile screens.
- **Backend API:** authentication, lobby lifecycle and real-time state over a persistent bidirectional connection, result validation, rating computation, leaderboards and contests.
- **Database:** users, prompt corpus, races, per-race results, rating history, contests and leaderboards.

### 5.2 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically type-check, lint, build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

## 6 Evaluation Criterion

*... measurable and attributable*

We define success using measurable metrics tied to the core workflow.

### 6.1 Primary evaluation metric

**Latency:** how quickly one player's progress becomes visible to the others in the same race.

- Target: median end-to-end update latency  $\leq 100$  ms during a race, reported alongside the 95th percentile.
- Attribution: each progress event carries a client send timestamp; the server records receive and broadcast timestamps and the receiving client records render time, so the delay is attributable to network, server processing or rendering rather than reported as one opaque number.

## 6.2 Secondary evaluation metrics

- **User experience:** number of actions and elapsed time from landing page to the start of a first race, measured for a new account.
- **Fair play:** on a set of deliberately scripted runs, the fraction correctly flagged, reported together with the false-positive rate on genuine human runs.
- **Rating quality:** how quickly a new player’s rating stabilises, measured as the race count after which rating change stays within a fixed band.
- **Service Level Agreements (SLA):** availability and response times sustained under a target number of concurrent races.

## 7 Scalability

*...with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in concurrent races and registered players by:
  - decoupling components (frontend/backend),
  - keeping ephemeral lobby state in a shared store so API instances stay horizontally scalable,
  - batching progress broadcasts instead of relaying every keystroke individually,
  - enabling pagination and indexing for leaderboard and history queries.
2. **Operationally deployable (practically):** the system should run on common web hosting:
  - managed database,
  - containerized or platform-hosted deployment,
  - CI/CD pipeline for staging/production.

## 8 Engine availability heuristic

A low latency set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework supporting both REST APIs and persistent bidirectional connections.
- Managed relational database for durable records.
- In-memory store for ephemeral lobby and matchmaking state.
- Authentication approach (token-based or session-based).
- Object storage for optional profile assets.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these low latency components to focus on real-time state design, correctness and measurement rather than inventing new infrastructure.

## 9 Project scope and deliverables

*... iteration-friendly*

### 9.1 Initial deliverable

*... first iteration*

- Single-player timed run on the existing typing surface, with WPM and accuracy computed from server-recorded start and finish timestamps.
- Account creation and sign-in.
- Structured logging and timestamps for latency measurement.
- CI: build + type-check + lint + backend unit tests.
- CD to staging with a single command or pipeline job.

### 9.2 Subsequent deliverables

- Two-player real-time race lobby with a shared prompt and live opponent progress.
- Rating computation and a global leaderboard.
- Rating-based matchmaking.
- Fair-play validation and a verified marker on qualifying results.
- Scheduled contests with standings and post-contest rating updates.
- Profile statistics and an activity heatmap.

## 10 Summary

This project proposes a deployable web platform on which users can practise typing, race skill-matched opponents, and obtain a speed figure that is computed and validated server-side rather than merely asserted. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially latency). It remains focused on engineering workflow and scalability readiness rather than research-driven novelty.